

JS Promises, Then catch, Async/Await- Practice Questions

1. Create a promise that resolves to "Hello, World!"

Expected Output:

```
"Hello, World!"
```

2. Create a promise-based function that returns the sum of two numbers

Expected Output:

```
30
```

3. Create a promise that rejects after 3 seconds

Expected Output (after 3 seconds):

```
"Rejected after 3 seconds"
```

4. Create a function that rejects if a string is empty

Expected Output:

```
"String cannot be empty"
```

5. Create a promise and log its state before and after resolving

Expected Output:

```
"Pending..."  
"Resolved!"
```

6. Create a function that logs a promise's state every secondExpected

Output:

```
"Pending..."  
"Pending..."  
"Resolved!"
```

7. Use `.catch()` to handle errors in promise chain

Expected Output:

```
"Success!" // If input is true  
"An error occurred" // If input is false
```

8. Handle errors using `try/catch` inside an `async` function

Expected Output:

```
"Error caught in try/catch"
```

9. Use `async/await` to return a squared number

Expected Output:

```
16
```

10. Use `Promise.all()` to resolve multiple promises

Expected Output:

```
["Result 1", "Result 2", "Result 3"]
```

JS Promises, Then catch, Async/Await- Assignment Questions

1. Create a function that rejects if the input number is odd

Expected Output:

```
"Number is odd"
```

2. Create a promise that retries three times before rejecting

Expected Output:

```
"Failed after 3 attempts"
```

3. Fetch data using `async/await`

Expected Output:

```
"Data received"
```

4. Use `Promise.allSettled()` to handle multiple outcomes

Expected Output:

```
[{ status: "fulfilled", value: "Success" }, { status: "rejected", reason: "Failed" }]
```

5. Create a function that retries a failed promise 3 times

Expected Output:

```
"Failed after 3 retries"
```